

DATA PROCESSING

Tools, Descriptive Analytics, and Data Analysis

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I



Outline

- 1 Tools and Process *W*
- 2 Descriptive Analytics *] R*
- 3 Data Analysis *] Rust*



Outline

- 1 Tools and Process
- 2 Descriptive Analytics
- 3 Data Analysis



VS Code and Extensions I

Visual Studio Code

VS Code is a **lightweight**, **extensible** source-code editor by **Microsoft** that supports multiple *languages*, *debuggers*, and *version control* workflows through a rich **extension marketplace**.

Key extensions for **Data Processing**:

- **Python** (ms-python.python): **IntelliSense**, **linting**, **debugging** and virtual environment management.
- **Data Wrangler**: interactive **pandas** DataFrame exploration and visual **data-cleaning** UI inside the notebook.
- **R** (REditorSupport.r): full **R language** support, integrated myemphplots, and R terminal execution.
- **Rust Analyzer** (rust-lang.rust-analyzer): **semantic analysis**, **code completion** and **inline diagnostics**.



VS Code and Extensions I

Visual Studio Code

VS Code is a **lightweight**, **extensible** source-code editor by **Microsoft** that supports multiple *languages*, *debuggers*, and *version control* workflows through a rich **extension marketplace**.

Key extensions for **Data Processing**:

- **Python** (ms-python.python): **IntelliSense**, **linting**, **debugging** and virtual environment management.
- **Data Wrangler**: interactive **pandas** DataFrame exploration and visual **data-cleaning** UI inside the notebook.
- **R** (REditorSupport.r): full **R language** support, integrated myemphplots, and R terminal execution.
- **Rust Analyzer** (rust-lang.rust-analyzer): **semantic analysis**, **code completion** and **inline diagnostics**.



VS Code and Extensions I

Visual Studio Code

VS Code is a **lightweight**, **extensible** source-code editor by **Microsoft** that supports multiple *languages*, *debuggers*, and *version control* workflows through a rich **extension marketplace**.

Key extensions for **Data Processing**:

- **Python** (ms-python.python): **IntelliSense**, **linting**, **debugging** and virtual environment management.
- **Data Wrangler**: interactive **pandas** DataFrame exploration and visual **data-cleaning** UI inside the notebook.
- **R** (REditorSupport.r): full **R language** support, integrated **myemphplots**, and R terminal execution.
- **Rust Analyzer** (rust-lang.rust-analyzer): **semantic analysis**, **code completion** and **inline diagnostics**.



VS Code and Extensions I

Visual Studio Code

VS Code is a **lightweight**, **extensible** source-code editor by **Microsoft** that supports multiple *languages*, *debuggers*, and *version control* workflows through a rich **extension marketplace**.

Key extensions for **Data Processing**:

- **Python** (ms-python.python): **IntelliSense**, **linting**, **debugging** and virtual environment management.
- **Data Wrangler**: interactive **pandas** DataFrame exploration and visual **data-cleaning** UI inside the notebook.
- **R** (REditorSupport.r): full **R language** support, integrated myemphplots, and R terminal execution.
- **Rust Analyzer** (rust-lang.rust-analyzer): **semantic analysis**, code completion and **inline diagnostics**.



VS Code and Extensions II

Key extensions for **Data Processing** (cont.):

- **Docker** (ms-azuretools.vscode-docker): manage **containers**, edit **Compose** files, and browse image registries.
- **Jupyter**: **notebook** execution, **variable inspector**, and kernel management without leaving the editor.



VS Code and Extensions II

Key extensions for **Data Processing** (cont.):

- **Docker** (ms-azuretools.vscode-docker): manage **containers**, edit **Compose** files, and browse image registries.
- **Jupyter**: **notebook** execution, **variable inspector**, and kernel management without leaving the editor.

Handwritten note in pink:

Julia
Python
R

|

Jo
Pyt
ek



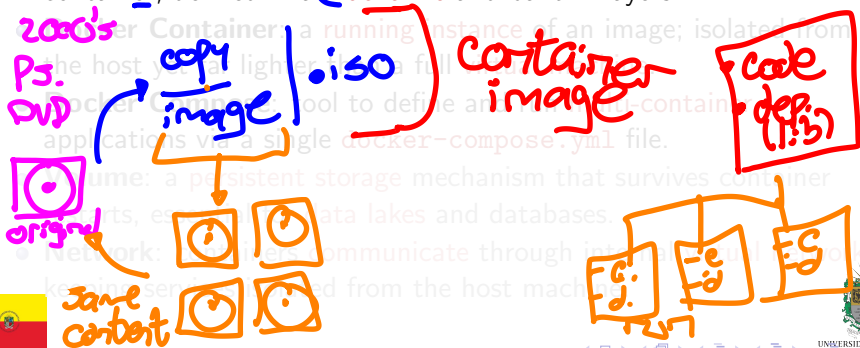
Docker and Containerization I

no installation

Container

A **container** is a **lightweight**, **portable**, and **self-sufficient unit** that packages **code** together with all its **dependencies**, so the application **runs identically** in any environment.

- **Docker Image**: a **read-only template** with instructions for creating a container, defined in a **Dockerfile** and built in layers.



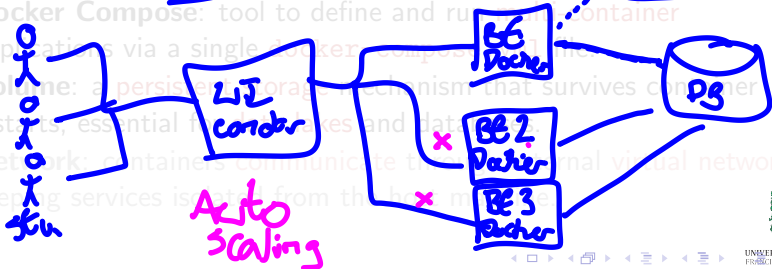
Docker and Containerization I

Container

A **container** is a **lightweight**, **portable**, and **self-sufficient unit** that packages **code** together with all its *dependencies*, so the application **runs identically** in any environment.

- **Docker Image**: a **read-only template** with instructions for creating a container, defined in a **Dockerfile** and built in layers.
- **Docker Container**: a **running instance** of an image; isolated from the host yet far lighter than a full **virtual machine**.

- **Docker Compose**: tool to define and run multi-container applications via a single **docker-compose.yml** file.
- **Volume**: a **persistent storage mechanism** that survives container restarts, essential for **databases** and **data**.
- **Network**: containers **communicate** through **internal virtual networks**, keeping services isolated from the host machine.



Docker and Containerization I

Container

A **container** is a **lightweight**, **portable**, and **self-sufficient unit** that packages **code** together with all its *dependencies*, so the application **runs identically** in any environment.

- **Docker Image**: a **read-only template** with instructions for creating a container, defined in a **Dockerfile** and built in layers.
- **Docker Container**: a **running instance** of an image; isolated from the host yet far lighter than a full **virtual machine**.
- **Docker Compose**: tool to define and run **multi-container** applications via a single **docker-compose.yml** file.
- **Volume**: a **persistent storage mechanism** that survives container restarts, essential for data and databases.
- **Network**: containers **communicate** through internal **virtual networks**, keeping services isolated from the host machine.



Docker and Containerization I

Container

A **container** is a **lightweight**, **portable**, and **self-sufficient unit** that packages **code** together with all its *dependencies*, so the application **runs identically** in any environment.

- **Docker Image**: a **read-only template** with instructions for creating a container, defined in a **Dockerfile** and built in layers.
- **Docker Container**: a **running instance** of an image; isolated from the host yet far lighter than a full **virtual machine**.
- **Docker Compose**: tool to define and run **multi-container** applications via a single **docker-compose.yml** file.
- **Volume**: a **persistent storage** mechanism that **survives container restarts**, essential for **data lakes** and databases.
- **Network**: containers **communicate** through internal **virtual networks**, keeping services isolated from the host machine.



Docker and Containerization I

Container

A **container** is a **lightweight**, **portable**, and **self-sufficient unit** that packages **code** together with all its *dependencies*, so the application **runs identically** in any environment.

- **Docker Image**: a **read-only template** with instructions for creating a container, defined in a **Dockerfile** and built in layers.
- **Docker Container**: a **running instance** of an image; isolated from the host yet far lighter than a full **virtual machine**.
- **Docker Compose**: tool to define and run **multi-container** applications via a single **docker-compose.yml** file.
- **Volume**: a **persistent storage** mechanism that survives container restarts, essential for **data lakes** and databases.
- **Network**: containers **communicate** through **internal virtual networks**, keeping services isolated from the host machine.



→ sub-networks



Docker and Containerization II

Practice: our **Airflow + PostgreSQL** pipeline uses 4 containers orchestrated with a single `docker compose up` command.



Data Sources: APIs and OpenData I

Application Programming Interface (API)

An **API** is a **contract** between systems that defines how **data** is **requested** and **delivered**, typically over HTTP using the *REST* style and *JSON* as the exchange format.

- **REST API**: uses **HTTP verbs** (GET, POST, ...) and **URL endpoints** to expose data resources.
- **OpenData**: publicly available datasets from **governments**, **universities**, and **research agencies** (data.gov, datos.gov.co, NASA).
- **NASA NEO Feed API**: returns **Near-Earth Objects** (asteroids) in a 7-day window — ≈ 150 objects per request.
 - Endpoint: /neo/rest/v1/feed?start_date=&end_date=&api_key=
 - Free key: DEMO_KEY (rate-limited to 30 req/hour)



Data Sources: APIs and OpenData I

Application Programming Interface (API)

An **API** is a **contract** between systems that defines how **data** is **requested** and **delivered**, typically over HTTP using the *REST* style and *JSON* as the exchange format.

- **REST API**: uses **HTTP verbs** (GET, POST, ...) and **URL endpoints** to expose data resources.
- **OpenData**: publicly available datasets from **governments**, **universities**, and **research agencies** (data.gov, datos.gov.co, NASA).
- **NASA NEO Feed API**: returns **Near-Earth Objects** (asteroids) in a 7-day window — ≈ 150 objects per request.
 - Endpoint: `/neo/rest/v1/feed?start_date=&end_date=&api_key=`
 - Free key: `DEMO_KEY` (rate-limited to **30 req/hour**)



Data Sources: APIs and OpenData I

Application Programming Interface (API)

An **API** is a **contract** between systems that defines how **data** is **requested** and **delivered**, typically over HTTP using the *REST* style and *JSON* as the exchange format.

- **REST API**: uses **HTTP verbs** (GET, POST, ...) and **URL endpoints** to expose data resources.
- **OpenData**: publicly available datasets from **governments**, **universities**, and **research agencies** (data.gov, datos.gov.co, NASA).
- **NASA NEO Feed API**: returns **Near-Earth Objects** (asteroids) in a 7-day window — ≈ 150 objects per request.
 - Endpoint: `/neo/rest/v1/feed?start_date=&end_date=&api_key=`
 - Free key: DEMO_KEY (rate-limited to **30 req/hour**)



Data Sources: APIs and OpenData II

- **Pagination:** large datasets require **iterating pages** via next links or offset parameters.
- **Rate Limiting:** APIs enforce **request quotas** per time window; robust pipelines handle **HTTP 429** responses gracefully.



Data Sources: APIs and OpenData II

- **Pagination:** large datasets require **iterating pages** via next links or offset parameters.
- **Rate Limiting:** APIs enforce **request quotas** per time window; robust pipelines handle **HTTP 429** responses gracefully.



Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that **extracts** data from sources, **transforms** it to meet quality and business rules, and **loads** it into a destination system ready for *analysis*.

- **Extract:** pull data from APIs, files, or databases into a staging area preserving the original format.
- **Transform:** clean, validate, deduplicate, and enrich data according to business rules.
- **Load:** persist transformed data into the target system (SQL, Parquet, NoSQL, or a data warehouse).

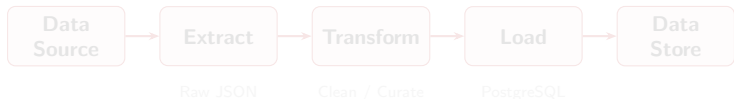


Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that **extracts** data from sources, **transforms** it to meet quality and business rules, and **loads** it into a destination system ready for *analysis*.

- **Extract:** pull data from APIs, files, or databases into a staging area preserving the original format.
- **Transform:** clean, validate, deduplicate, and enrich data according to business rules.
- **Load:** persist transformed data into the target system (SQL, Parquet, NoSQL, or a data warehouse).

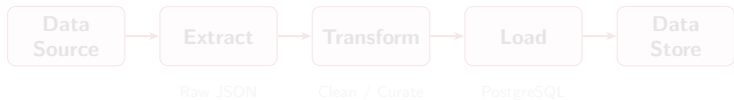


Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that **extracts** data from sources, **transforms** it to meet quality and business rules, and **loads** it into a destination system ready for *analysis*.

- **Extract:** **pull data** from APIs, files, or databases into a **staging area** preserving the original format.
- **Transform:** **clean, validate, deduplicate, and enrich** data according to business rules.
- **Load:** **persist** transformed data into the **target system** (SQL, Parquet, NoSQL, or a data warehouse).

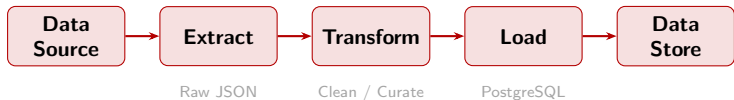


Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that **extracts** data from sources, **transforms** it to meet quality and business rules, and **loads** it into a destination system ready for *analysis*.

- **Extract:** **pull data** from APIs, files, or databases into a **staging area** preserving the original format.
- **Transform:** **clean, validate, deduplicate, and enrich** data according to business rules.
- **Load:** **persist** transformed data into the **target system** (SQL, Parquet, NoSQL, or a data warehouse).



Apache Airflow for Data Pipelines

DAG — Directed Acyclic Graph

In Airflow, a **DAG** defines the **order** and **dependencies** of **tasks** in a pipeline. Each *node* is a task; *edges* enforce execution order and guarantee **no cycles**.



- **Scheduler:** monitors DAGs and triggers task runs based on cron expressions (e.g., `*/10 * * * *`).

- **Operators:** task types — `FileOperator`, `PythonOperator`, `PostgresOperator`, etc.

- **XCom:** mechanism for inter-task communication; tasks can store small values between steps.



Apache Airflow for Data Pipelines

DAG — Directed Acyclic Graph

In Airflow, a **DAG** defines the **order** and **dependencies** of **tasks** in a pipeline. Each *node* is a task; *edges* enforce execution order and guarantee **no cycles**.



- **Scheduler:** monitors DAGs and **triggers** task runs based on **cron expressions** (e.g., `*/10 * * * *`).
- **Operators:** task types — `PythonOperator`, `PostgresOperator`, `TriggerDagRunOperator`, etc.
- **XCom:** mechanism for **inter-task** communication; tasks push and pull small values between steps.



Apache Airflow for Data Pipelines

DAG — Directed Acyclic Graph

In Airflow, a **DAG** defines the **order** and **dependencies** of **tasks** in a pipeline. Each *node* is a task; *edges* enforce execution order and guarantee **no cycles**.



- **Scheduler:** monitors DAGs and **triggers** task runs based on **cron expressions** (e.g., `*/10 * * * *`).
- **Operators:** task types — `PythonOperator`, `PostgresOperator`, `TriggerDagRunOperator`, etc.
- **XCom:** mechanism for **inter-task** communication; tasks **push** and **pull** small values between steps.



Apache Airflow for Data Pipelines

DAG — Directed Acyclic Graph

In Airflow, a **DAG** defines the **order** and **dependencies** of **tasks** in a pipeline. Each *node* is a task; *edges* enforce execution order and guarantee **no cycles**.



- **Scheduler:** monitors DAGs and **triggers** task runs based on **cron expressions** (e.g., `*/10 * * * *`).
- **Operators:** task types — **PythonOperator**, **PostgresOperator**, **TriggerDagRunOperator**, etc.
- **XCom:** mechanism for **inter-task** communication; tasks **push** and **pull** small values between steps.



Apache Airflow for Data Pipelines

DAG — Directed Acyclic Graph

In Airflow, a **DAG** defines the **order** and **dependencies** of **tasks** in a pipeline. Each *node* is a task; *edges* enforce execution order and guarantee **no cycles**.



- **Scheduler:** monitors DAGs and **triggers** task runs based on **cron expressions** (e.g., `*/10 * * * *`).
- **Operators:** task types — **PythonOperator**, **PostgresOperator**, **TriggerDagRunOperator**, etc.
- **XCom:** mechanism for **inter-task** communication; tasks **push** and **pull** small values between steps.



Apache Parquet and Tabular Formats I

Parquet

Apache Parquet is an **open-source**, **columnar** storage format designed for **efficient compression** and **encoding schemes**, optimized for **analytical queries** (OLAP) on large-scale datasets.

Columnar advantages:

- **Compression** up to $4\times$ – $10\times$ vs. row formats.
- **Predicate pushdown**: skip irrelevant **row groups** at read time.
- Only load the **selected columns** — no full table scan.
- **Schema enforcement** prevents bad data from entering the lake.



Apache Parquet and Tabular Formats II

Format comparison:

- **CSV**: human-readable, no schema, **row-oriented**.
- **JSON**: flexible, verbose, **schema-free**.
- **Parquet**: binary, schema-enforced, **column-oriented**.
- **ORC**: columnar like Parquet, optimized for **Hive/Spark**.



Datalakes and Data Ingestion I

Data Lake

A **Data Lake** is a **centralized repository** that stores **raw data** at any **scale** — structured, semi-structured, and **unstructured** — in its *native format* until it is needed for analysis or modelling.

- **Data Ingestion:** process of **importing**, **transferring**, and **loading** data from diverse sources into a storage system.
- **Batch Ingestion:** data collected in **scheduled intervals** (e.g., every 10 minutes via an **Airflow DAG**).
- **Stream Ingestion:** data **continuously ingested** in real time using tools like **Apache Kafka** or **AWS Kinesis**.



Datalakes and Data Ingestion I

Data Lake

A **Data Lake** is a **centralized repository** that stores **raw data** at any **scale** — structured, semi-structured, and **unstructured** — in its *native format* until it is needed for analysis or modelling.

- **Data Ingestion:** process of **importing**, **transferring**, and **loading** data from diverse sources into a storage system.
- **Batch Ingestion:** data collected in **scheduled intervals** (e.g., every 10 minutes via an **Airflow DAG**).
- **Stream Ingestion:** data **continuously ingested** in real time using tools like **Apache Kafka** or **AWS Kinesis**.



Datalakes and Data Ingestion I

Data Lake

A **Data Lake** is a **centralized repository** that stores **raw data** at any **scale** — structured, semi-structured, and **unstructured** — in its *native format* until it is needed for analysis or modelling.

- **Data Ingestion:** process of **importing**, **transferring**, and **loading** data from diverse sources into a storage system.
- **Batch Ingestion:** data collected in **scheduled intervals** (e.g., every 10 minutes via an **Airflow DAG**).
- **Stream Ingestion:** data **continuously ingested** in real time using tools like **Apache Kafka** or **AWS Kinesis**.



Datalakes and Data Ingestion I

Data Lake

A **Data Lake** is a **centralized repository** that stores **raw data** at any **scale** — structured, semi-structured, and **unstructured** — in its *native format* until it is needed for analysis or modelling.

- **Data Ingestion:** process of **importing**, **transferring**, and **loading** data from diverse sources into a storage system.
- **Batch Ingestion:** data collected in **scheduled intervals** (e.g., every 10 minutes via an **Airflow DAG**).
- **Stream Ingestion:** data **continuously ingested** in real time using tools like **Apache Kafka** or **AWS Kinesis**.



Datalakes and Data Ingestion II

- **Schema-on-Read:** the **schema** is applied at **query time**, not at write time — enabling **flexibility** in raw storage.
- **Partitioning:** organizing files by **date** or **category** eliminates irrelevant files and **speeds up queries** significantly.



Datalakes and Data Ingestion II

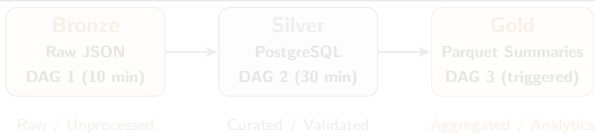
- **Schema-on-Read:** the **schema** is applied at **query time**, not at write time — enabling **flexibility** in raw storage.
- **Partitioning:** organizing files by **date** or **category** eliminates irrelevant files and **speeds up queries** significantly.



Datalake Medallion Architecture

Medallion Architecture

The **Medallion Architecture** organizes a datalake into **three layers** – *Bronze*, *Silver*, and *Gold* – that progressively **refine data quality** from raw ingestion to analytical consumption.



- **Bronze**: raw ingestion, no transformations, preserves the original data exactly as received.
- **Silver**: cleaned, deduplicated, and validated records loaded into structured storage (SQL).
- **Gold**: aggregated, optimized for BI tools, dashboards, and reports. Stored as Parquet.



Datalake Medallion Architecture

Medallion Architecture

The **Medallion Architecture** organizes a datalake into **three layers** – *Bronze*, *Silver*, and *Gold* – that progressively **refine data quality** from raw ingestion to analytical consumption.



- **Bronze:** raw ingestion, no transformations, preserves the original data exactly as received.
- **Silver:** cleaned, deduplicated, and validated records loaded into structured storage (SQL).
- **Gold:** aggregated summaries optimized for BI tools, dashboards, and ML models. Stored as Parquet.



Datalake Medallion Architecture

Medallion Architecture

The **Medallion Architecture** organizes a datalake into **three layers** – *Bronze*, *Silver*, and *Gold* – that progressively **refine data quality** from raw ingestion to analytical consumption.



- **Bronze:** **raw ingestion**, no transformations, preserves the **original data** exactly as received.
- **Silver:** **cleaned, deduplicated, and validated** records loaded into structured storage (SQL).
- **Gold:** **aggregated summaries** optimized for **BI tools, dashboards, and ML models**. Stored as Parquet.



Datalake Medallion Architecture

Medallion Architecture

The **Medallion Architecture** organizes a datalake into **three layers** – *Bronze*, *Silver*, and *Gold* – that progressively **refine data quality** from raw ingestion to analytical consumption.



- **Bronze:** **raw ingestion**, no transformations, preserves the **original data** exactly as received.
- **Silver:** **cleaned**, **deduplicated**, and **validated** records loaded into structured storage (SQL).
- **Gold:** **aggregated summaries** optimized for BI tools, dashboards, and ML models. Stored as Parquet.



Datalake Medallion Architecture

Medallion Architecture

The **Medallion Architecture** organizes a datalake into **three layers** – *Bronze*, *Silver*, and *Gold* – that progressively **refine data quality** from raw ingestion to analytical consumption.



- **Bronze:** **raw ingestion**, no transformations, preserves the **original data** exactly as received.
- **Silver:** **cleaned**, **deduplicated**, and **validated** records loaded into structured storage (SQL).
- **Gold:** **aggregated summaries** optimized for **BI tools**, dashboards, and **ML models**. Stored as Parquet.



Outline

- 1 Tools and Process
- 2 Descriptive Analytics
- 3 Data Analysis



R for Descriptive Statistics

R Language

R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

Core capabilities:

- Vectors and data frames as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- `ggplot2` for layer-based, publication-quality

Key packages:

- `tidyverse` ecosystem
- `shiny` for interactive web applications
- `tidyr`, `readr`, and `writer` for data wrangling
- `psych` for psychological data analysis
- `car` for advanced regression analysis



R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

- **Vectors** and **data frames** as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- **ggplot2** for layer-based, publication-quality **plots**.



R for Descriptive Statistics

R Language

R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

Core capabilities:

- **Vectors** and **data frames** as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- **ggplot2** for layer-based, publication-quality **plots**.

Key packages:

- **dplyr**: data manipulation and filtering grammar.
- **tidyr**: tidying and reshaping data frames.
- **psych**: psychological data analysis, correlation, and factor analysis.
- **caret** / **tidymodels**: machine learning workflows and evaluation pipelines.



R for Descriptive Statistics

R Language

R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

Core capabilities:

- **Vectors** and **data frames** as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- **ggplot2** for layer-based, publication-quality **plots**.

Key packages:

- **dplyr**: data manipulation and filtering grammar.
- **tidyr**: reshaping and pivoting data frames.
- **psych**: psychological data analysis, correlation, and factor analysis.
- **caret** / **tidymodels**: machine learning workflows and evaluation pipelines.



R for Descriptive Statistics

R Language

R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

Core capabilities:

- **Vectors** and **data frames** as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- **ggplot2** for layer-based, publication-quality **plots**.

Key packages:

- **dplyr**: **data manipulation** and **filtering** grammar.
- **tidyr**: **reshaping** and **pivoting** data frames.
- **psych**: **descriptive stats**, **correlation**, and **factor analysis**.
- **caret** / **tidymodels**: **ML model training** and **evaluation pipelines**.



R for Descriptive Statistics

R Language

R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

Core capabilities:

- **Vectors** and **data frames** as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- **ggplot2** for layer-based, publication-quality **plots**.

Key packages:

- **dplyr**: **data manipulation** and **filtering** grammar.
- **tidyr**: **reshaping** and **pivoting** data frames.
- **psych**: **descriptive stats**, **correlation**, and **factor analysis**.
- **caret** / **tidymodels**: **ML model training** and **evaluation pipelines**.



R for Descriptive Statistics

R Language

R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

Core capabilities:

- **Vectors** and **data frames** as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- **ggplot2** for layer-based, publication-quality **plots**.

Key packages:

- **dplyr**: **data manipulation** and **filtering** grammar.
- **tidyr**: **reshaping** and **pivoting** data frames.
- **psych**: **descriptive stats**, **correlation**, and **factor analysis**.
- **caret** / **tidymodels**: **ML model training** and **evaluation pipelines**.



R for Descriptive Statistics

R Language

R is a **statistical programming language** designed for **data analysis**, **visualization**, and **statistical computing**, widely used in *academia*, *biostatistics*, and *data science*.

Core capabilities:

- **Vectors** and **data frames** as native first-class types.
- Built-in functions: `mean()`, `sd()`, `summary()`, `cor()`, and `lm()`.
- **ggplot2** for layer-based, publication-quality **plots**.

Key packages:

- **dplyr**: **data manipulation** and **filtering** grammar.
- **tidyr**: **reshaping** and **pivoting** data frames.
- **psych**: **descriptive stats**, **correlation**, and **factor analysis**.
- **caret** / **tidymodels**: **ML model training** and evaluation pipelines.



Data Composition I

Data Types in a Dataset

Understanding **data composition** means identifying **what variable types** exist and **how they are distributed**, which determines which *statistical methods* are applicable.



Data Composition II

- **Numerical (Quantitative):**

- **Continuous:** any real value in a range (velocity, distance, temperature).
- **Discrete:** countable integers (asteroid count, events per day).

- **Categorical (Qualitative):**

- **Nominal:** no order (country, orbiting body name).
- **Ordinal:** ordered categories (low / medium / high hazard level).

- **Temporal:** time-series data with date/time indices; requires specific handling for trend and seasonality.
- **Boolean:** binary flags, e.g., is_potentially_hazardous.
- **Missing:** NaN / null values that must be identified and handled before any statistical analysis.



Data Composition II

- **Numerical (Quantitative):**

- **Continuous:** any real value in a range (velocity, distance, temperature).
- **Discrete:** countable integers (asteroid count, events per day).

- **Categorical (Qualitative):**

- **Nominal:** no order (country, orbiting body name).
- **Ordinal:** ordered categories (low / medium / high hazard level).
- **Temporal:** time-series data with date/time indices; requires specific handling for trend and seasonality.
- **Boolean:** binary flags, e.g., is_potentially_hazardous.
- **Missing:** NaN / null values that must be identified and handled before any statistical analysis.



Data Composition II

- **Numerical (Quantitative):**

- **Continuous:** any real value in a range (velocity, distance, temperature).
- **Discrete:** countable integers (asteroid count, events per day).

- **Categorical (Qualitative):**

- **Nominal:** no order (country, orbiting body name).
- **Ordinal:** ordered categories (low / medium / high hazard level).

- **Temporal:** **time-series** data with **date/time** indices; requires specific handling for trend and seasonality.

- **Boolean:** **binary flags**, e.g., `is_potentially_hazardous`.

- **Missing:** **NaN** / **null** values that must be identified and handled **before** any statistical analysis.



Data Composition II

- **Numerical (Quantitative):**

- **Continuous:** any real value in a range (velocity, distance, temperature).
- **Discrete:** countable integers (asteroid count, events per day).

- **Categorical (Qualitative):**

- **Nominal:** no order (country, orbiting body name).
- **Ordinal:** ordered categories (low / medium / high hazard level).

- **Temporal:** **time-series** data with **date/time** indices; requires specific handling for trend and seasonality.

- **Boolean:** **binary flags**, e.g., `is_potentially_hazardous`.

- **Missing:** **NaN / null** values that must be identified and handled **before** any statistical analysis.



Data Composition II

- **Numerical (Quantitative):**
 - **Continuous:** any real value in a range (velocity, distance, temperature).
 - **Discrete:** countable integers (asteroid count, events per day).
- **Categorical (Qualitative):**
 - **Nominal:** no order (country, orbiting body name).
 - **Ordinal:** ordered categories (low / medium / high hazard level).
- **Temporal:** **time-series** data with **date/time** indices; requires specific handling for trend and seasonality.
- **Boolean:** **binary flags**, e.g., `is_potentially_hazardous`.
- **Missing:** **NaN** / **null** values that must be identified and handled **before** any statistical analysis.



Mean and Median

Mean (Arithmetic Average)

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Sensitive to **outliers**; best for **symmetric** distributions.

Median

The **middle value** when data is sorted in ascending order.

Robust to outliers; preferred for **skewed** distributions.



Mean and Median

Mean (Arithmetic Average)

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Sensitive to **outliers**; best for **symmetric** distributions.

Median

The **middle value** when data is sorted in ascending order.

Robust to outliers; preferred for **skewed** distributions.



Mode and Range

Mode

The **most frequent** value in the dataset.

Useful for **categorical** and **discrete** data; a dataset can be **unimodal**, **bimodal**, or **multimodal**.

Range

$$R = x_{\max} - x_{\min}$$

Simplest measure of spread; **highly sensitive** to **extreme values**.



Mode and Range

Mode

The **most frequent** value in the dataset.

Useful for **categorical** and **discrete** data; a dataset can be **unimodal**, **bimodal**, or **multimodal**.

Range

$$R = x_{\max} - x_{\min}$$

Simplest measure of spread; **highly sensitive** to **extreme values**.



Normal Data, Standard Deviation, and Variance

Normal (Gaussian) Distribution

A **symmetric**, **bell-shaped** continuous probability distribution fully described by its **mean** μ and **standard deviation** σ :

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x - \mu}{\sigma}\right)^2\right)$$

Variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Average squared deviation from the mean; in squared units.

Standard Deviation

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Same units as data; measures dispersion around $\bar{x}$.

Normal Data, Standard Deviation, and Variance

Normal (Gaussian) Distribution

A **symmetric**, **bell-shaped** continuous probability distribution fully described by its **mean** μ and **standard deviation** σ :

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

Variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Average squared deviation from the mean; in **squared units**.

Standard Deviation

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Same units as data; measures dispersion around $\bar{x}$.

68-95-99.7 rule: $\approx 68\%$ of data within $\pm 1\sigma$, 95% within $\pm 2\sigma$, 99.7% within $\pm 3\sigma$

Normal Data, Standard Deviation, and Variance

Normal (Gaussian) Distribution

A **symmetric**, **bell-shaped** continuous probability distribution fully described by its **mean** μ and **standard deviation** σ :

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x - \mu}{\sigma}\right)^2\right)$$

Variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Average squared deviation from the mean; in **squared units**.

Standard Deviation

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Same units as data; measures **dispersion** around $\bar{x}$.

68-95-99.7 rule: $\approx 68\%$ of data within $\pm 1\sigma$, 95% within $\pm 2\sigma$, 99.7% within $\pm 3\sigma$

Normal Data, Standard Deviation, and Variance

Normal (Gaussian) Distribution

A **symmetric**, **bell-shaped** continuous probability distribution fully described by its **mean** μ and **standard deviation** σ :

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x - \mu}{\sigma}\right)^2\right)$$

Variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Average squared deviation from the mean; in **squared units**.

Standard Deviation

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Same units as data; measures **dispersion** around $\bar{x}$.

68-95-99.7 rule: $\approx 68\%$ of data within $\pm 1\sigma$, 95% within $\pm 2\sigma$, 99.7% within $\pm 3\sigma$.

Skewness and Kurtosis

Skewness

$$\gamma_1 = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3$$

- $\gamma_1 = 0$: **symmetric** (normal).
- $\gamma_1 > 0$: **right-skewed**, long right tail.
- $\gamma_1 < 0$: **left-skewed**, long left tail.

Excess Kurtosis

$$\gamma_2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^4 - 3$$

- $\gamma_2 = 0$: **mesokurtic** (normal tails).
- $\gamma_2 > 0$: **leptokurtic**, heavy tails, peaked.
- $\gamma_2 < 0$: **platykurtic**, light tails, flat.

High **skewness** signals **outliers** or **non-normality**; high **kurtosis** indicates **extreme events** more frequent than in a normal distribution — critical for **financial and risk analysis**.

Skewness and Kurtosis

Skewness

$$\gamma_1 = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3$$

- $\gamma_1 = 0$: **symmetric** (normal).
- $\gamma_1 > 0$: **right-skewed**, long right tail.
- $\gamma_1 < 0$: **left-skewed**, long left tail.

Excess Kurtosis

$$\gamma_2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^4 - 3$$

- $\gamma_2 = 0$: **mesokurtic** (normal tails).
- $\gamma_2 > 0$: **leptokurtic**, heavy tails, peaked.
- $\gamma_2 < 0$: **platykurtic**, light tails, flat.

High **skewness** signals **outliers** or **non-normality**; high **kurtosis** indicates **extreme events** more frequent than in a normal distribution — critical for **financial** and **risk analysis**.

Histograms and Data Distribution I

Histogram

A **histogram** divides data into contiguous **bins** and plots the **frequency** or **density** per bin, revealing the *shape*, *spread*, and *modality* of the data distribution.



Histograms and Data Distribution [II]

- **Bin width:** too narrow $\Rightarrow$ **noisy**; too wide $\Rightarrow$ **over-smoothed**.
Sturges' rule: $k = \lceil 1 + \log_2 n \rceil$ bins for n observations.
- **Unimodal:** **one peak** — possibly normal. **Bimodal:** **two peaks** — suggests a **mixture** of populations.
- **Density plot (KDE):** smooth **continuous estimate** of distribution using a **kernel function**; area under curve = 1.
- **Frequency vs. Density:**
 - **Frequency:** raw count of observations per bin.
 - **Density:** count $\div$ (total $\times$ bin width) — sums to 1.
- **Interpretation:** shape reveals **skewness**, **outliers**, and whether **normality** is a reasonable assumption.



Histograms and Data Distribution [II]

- **Bin width:** too narrow $\Rightarrow$ **noisy**; too wide $\Rightarrow$ **over-smoothed**.
Sturges' rule: $k = \lceil 1 + \log_2 n \rceil$ bins for n observations.
- **Unimodal:** **one peak** — possibly normal. **Bimodal:** **two peaks** — suggests a **mixture** of populations.
- **Density plot (KDE):** smooth **continuous estimate** of distribution using a **kernel function**; area under curve = 1.
- **Frequency vs. Density:**
 - **Frequency:** raw count of observations per bin.
 - **Density:** count $\div$ (total $\times$ bin width) — sums to 1.
- **Interpretation:** shape reveals **skewness**, **outliers**, and whether **normality** is a reasonable assumption.



Histograms and Data Distribution [II]

- **Bin width:** too narrow $\Rightarrow$ **noisy**; too wide $\Rightarrow$ **over-smoothed**.
Sturges' rule: $k = \lceil 1 + \log_2 n \rceil$ bins for n observations.
- **Unimodal:** **one peak** — possibly normal. **Bimodal:** **two peaks** — suggests a **mixture** of populations.
- **Density plot (KDE):** smooth **continuous estimate** of distribution using a **kernel function**; area under curve = 1.
- **Frequency vs. Density:**
 - **Frequency:** raw count of observations per bin.
 - **Density:** count $\div$ (total $\times$ bin width) — sums to 1.
- **Interpretation:** shape reveals **skewness**, **outliers**, and whether **normality** is a reasonable assumption.



Histograms and Data Distribution [II]

- **Bin width:** too narrow $\Rightarrow$ **noisy**; too wide $\Rightarrow$ **over-smoothed**.
Sturges' rule: $k = \lceil 1 + \log_2 n \rceil$ bins for n observations.
- **Unimodal:** **one peak** — possibly normal. **Bimodal:** **two peaks** — suggests a **mixture** of populations.
- **Density plot (KDE):** smooth **continuous estimate** of distribution using a **kernel function**; area under curve = 1.
- **Frequency vs. Density:**
 - **Frequency:** raw count of observations per bin.
 - **Density:** count $\div$ (total $\times$ bin width) — sums to 1.
- **Interpretation:** shape reveals **skewness**, **outliers**, and whether **normality** is a reasonable assumption.



Histograms and Data Distribution [II]

- **Bin width:** too narrow $\Rightarrow$ **noisy**; too wide $\Rightarrow$ **over-smoothed**.
Sturges' rule: $k = \lceil 1 + \log_2 n \rceil$ bins for n observations.
- **Unimodal:** **one peak** — possibly normal. **Bimodal:** **two peaks** — suggests a **mixture** of populations.
- **Density plot (KDE):** smooth **continuous estimate** of distribution using a **kernel function**; area under curve = 1.
- **Frequency vs. Density:**
 - **Frequency:** raw count of observations per bin.
 - **Density:** count $\div$ (total $\times$ bin width) — sums to 1.
- **Interpretation:** shape reveals **skewness**, **outliers**, and whether **normality** is a reasonable assumption.

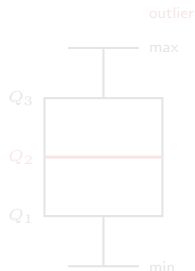


BoxPlots and Data Confidence I

BoxPlot (Box-and-Whisker)

A **boxplot** summarizes a distribution with **five statistics**: min, Q_1 , Q_2 (median), Q_3 , max, and visually exposes **outliers** and **distributional spread**.

- Q_1 = 25th percentile; Q_3 = 75th percentile.
- $IQR = Q_3 - Q_1$: the interquartile range (box width).
- Whiskers: extend to $Q_1 - 1.5 \cdot IQR$ and $Q_3 + 1.5 \cdot IQR$.
- Points outside whiskers are flagged as outliers.
- Notched boxplot: notch represents the confidence interval around the median, useful for comparing medians.

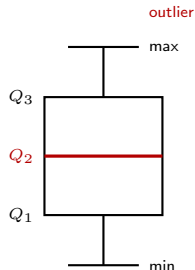


BoxPlots and Data Confidence I

BoxPlot (Box-and-Whisker)

A **boxplot** summarizes a distribution with **five statistics**: min, Q_1 , Q_2 (median), Q_3 , max, and visually exposes **outliers** and **distributional spread**.

- $Q_1 = 25\text{th percentile}$; $Q_3 = 75\text{th percentile}$.
- $IQR = Q_3 - Q_1$: the **interquartile range** (box width).
- **Whiskers**: extend to $Q_1 - 1.5 \cdot IQR$ and $Q_3 + 1.5 \cdot IQR$.
- Points **outside whiskers** are flagged as **outliers**.
- **Notched boxplot**: notch represents the 95% confidence interval around the median, useful for group comparisons.

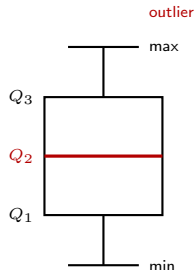


BoxPlots and Data Confidence I

BoxPlot (Box-and-Whisker)

A **boxplot** summarizes a distribution with **five statistics**: $\min$, Q_1 , Q_2 (median), Q_3 , $\max$, and visually exposes **outliers** and **distributional spread**.

- $Q_1 = 25\text{th percentile}$; $Q_3 = 75\text{th percentile}$.
- $IQR = Q_3 - Q_1$: the **interquartile range** (box width).
- **Whiskers**: extend to $Q_1 - 1.5 \cdot IQR$ and $Q_3 + 1.5 \cdot IQR$.
- Points **outside whiskers** are flagged as **outliers**.
- **Notched boxplot**: notch represents the **95% confidence interval** around the **median**, useful for **group comparisons**.

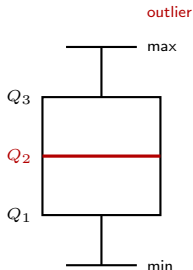


BoxPlots and Data Confidence I

BoxPlot (Box-and-Whisker)

A **boxplot** summarizes a distribution with **five statistics**: $\min$, Q_1 , Q_2 (median), Q_3 , $\max$, and visually exposes **outliers** and **distributional spread**.

- $Q_1 = 25\text{th percentile}$; $Q_3 = 75\text{th percentile}$.
- $IQR = Q_3 - Q_1$: the **interquartile range** (box width).
- **Whiskers**: extend to $Q_1 - 1.5 \cdot IQR$ and $Q_3 + 1.5 \cdot IQR$.
- Points **outside whiskers** are flagged as **outliers**.
- **Notched boxplot**: notch represents the **95% confidence interval** around the **median**, useful for **group comparisons**.

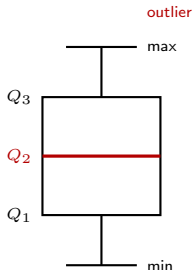


BoxPlots and Data Confidence I

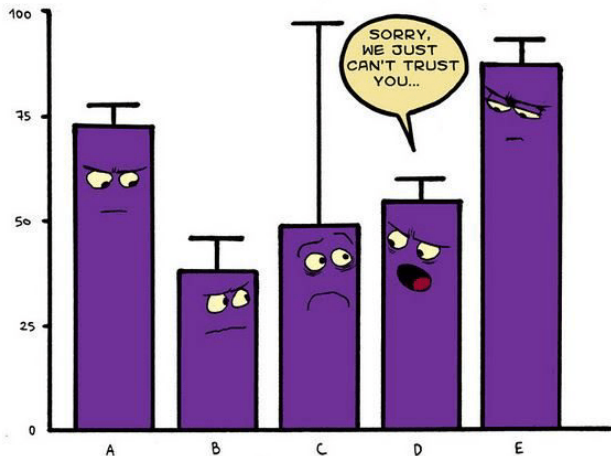
BoxPlot (Box-and-Whisker)

A **boxplot** summarizes a distribution with **five statistics**: $\min$, Q_1 , Q_2 (median), Q_3 , $\max$, and visually exposes **outliers** and **distributional spread**.

- $Q_1 = 25\text{th percentile}$; $Q_3 = 75\text{th percentile}$.
- $IQR = Q_3 - Q_1$: the **interquartile range** (box width).
- **Whiskers**: extend to $Q_1 - 1.5 \cdot IQR$ and $Q_3 + 1.5 \cdot IQR$.
- Points **outside whiskers** are flagged as **outliers**.
- **Notched boxplot**: notch represents the **95% confidence interval** around the **median**, useful for **group comparisons**.



BoxPlots and Data Confidence II



Correlation Analysis

Correlation

Correlation quantifies the **strength** and **direction** of the **statistical relationship** between two variables. It describes *association*, not **causation**.

- **Pearson** $r \in [-1, 1]$: measures **linear** correlation between **continuous, normally distributed** variables.
- **Spearman** ρ : measures **monotonic** relationships; rank-based and robust to outliers.
- **Kendall** τ : based on **concordant / discordant** pairs; preferred for small samples or many ties.
- **Correlation heatmap**: matrix view of pairwise correlations across all variables, each cell color-coded by magnitude.
- **Interpretation guide**:
 - $|r| \geq 0.7$ — **strong** correlation.
 - $0.4 \leq |r| < 0.7$ — **moderate** correlation.
 - $|r| < 0.4$ — **weak or negligible** correlation.



Correlation Analysis

Correlation

Correlation quantifies the **strength** and **direction** of the **statistical relationship** between two variables. It describes *association*, not **causation**.

- **Pearson** $r \in [-1, 1]$: measures **linear** correlation between **continuous, normally distributed** variables.
- **Spearman** ρ : measures **monotonic** relationships; **rank-based** and robust to **outliers**.
- **Kendall** τ : based on **concordant / discordant pairs**; preferred for **small samples** or many ties.
- **Correlation heatmap**: matrix view of **pairwise coefficients** across all variables, each cell color-coded by magnitude.
- **Interpretation guide**:
 - $|r| \geq 0.7$ — **strong** correlation.
 - $0.4 \leq |r| < 0.7$ — **moderate** correlation.
 - $|r| < 0.4$ — **weak or negligible** correlation.



Correlation Analysis

Correlation

Correlation quantifies the **strength** and **direction** of the **statistical relationship** between two variables. It describes *association*, not **causation**.

- **Pearson** $r \in [-1, 1]$: measures **linear** correlation between **continuous, normally distributed** variables.
- **Spearman** ρ : measures **monotonic** relationships; **rank-based** and robust to **outliers**.
- **Kendall** τ : based on **concordant / discordant pairs**; preferred for **small samples** or many ties.
- **Correlation heatmap**: matrix view of **pairwise coefficients** across all variables, each cell color-coded by magnitude.
- **Interpretation guide**:
 - $|r| \geq 0.7$ — **strong** correlation.
 - $0.4 \leq |r| < 0.7$ — **moderate** correlation.
 - $|r| < 0.4$ — **weak** or negligible correlation.



Correlation Analysis

Correlation

Correlation quantifies the **strength** and **direction** of the **statistical relationship** between two variables. It describes *association*, not **causation**.

- **Pearson** $r \in [-1, 1]$: measures **linear** correlation between **continuous, normally distributed** variables.
- **Spearman** ρ : measures **monotonic** relationships; **rank-based** and robust to **outliers**.
- **Kendall** τ : based on **concordant / discordant pairs**; preferred for **small samples** or many ties.
- **Correlation heatmap**: matrix view of **pairwise coefficients** across all variables, each cell color-coded by magnitude.
- **Interpretation guide**:
 - $|r| \geq 0.7$ — **strong** correlation.
 - $0.4 \leq |r| < 0.7$ — **moderate** correlation.
 - $|r| < 0.4$ — **weak** or negligible correlation.



Correlation Analysis

Correlation

Correlation quantifies the **strength** and **direction** of the **statistical relationship** between two variables. It describes *association*, not **causation**.

- **Pearson** $r \in [-1, 1]$: measures **linear** correlation between **continuous, normally distributed** variables.
- **Spearman** ρ : measures **monotonic** relationships; **rank-based** and robust to **outliers**.
- **Kendall** τ : based on **concordant / discordant pairs**; preferred for **small samples** or many ties.
- **Correlation heatmap**: matrix view of **pairwise coefficients** across all variables, each cell color-coded by magnitude.
- **Interpretation guide**:
 - $|r| \geq 0.7$ — **strong** correlation.
 - $0.4 \leq |r| < 0.7$ — **moderate** correlation.
 - $|r| < 0.4$ — **weak** or negligible correlation.

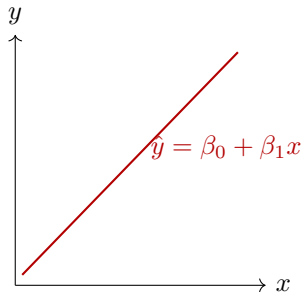


Tendency Analysis: Linear Regression I

Simple Linear Regression

Linear regression models the **linear relationship** between a **dependent variable** y and an **independent variable** x :

$$\hat{y} = \beta_0 + \beta_1 x$$



Tendency Analysis: Linear Regression II

OLS Coefficient Estimates:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

Goodness of Fit:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}, \quad SS_{\text{res}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Tendency Analysis: Linear Regression II

OLS Coefficient Estimates:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

Goodness of Fit:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}, \quad SS_{\text{res}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Outline

- 1 Tools and Process
- 2 Descriptive Analytics
- 3 Data Analysis



Rust for Data Analysis I

Rust Language

Rust is a **systems programming language** focused on **memory safety**, **zero-cost abstractions**, and **performance**, achieving **C-level speed** without a garbage collector via its *ownership model*.

Why Rust for data?

- **No runtime overhead**: ideal for **large-scale** data crunching.
- **Fearless concurrency**: safe **parallel** processing with no data races guaranteed at compile time.
- **Polars** core is Rust; Python bindings share the **same speed**.



Rust for Data Analysis I

Rust Language

Rust is a **systems programming language** focused on **memory safety**, **zero-cost abstractions**, and **performance**, achieving **C-level speed** without a garbage collector via its *ownership model*.

Why Rust for data?

- **No runtime overhead**: ideal for **large-scale** data crunching.
- **Fearless concurrency**: safe **parallel** processing with no data races guaranteed at compile time.
- **Polars** core is Rust; Python bindings share the **same speed**.



Rust for Data Analysis II

Key crates (libraries):

- polars: fast **DataFrame** library backed by **Arrow**.
- arrow2: **Apache Arrow** in-memory columnar format.
- ndarray: **N-dimensional arrays** similar to **NumPy**.
- rayon: **data-parallel** iterators with work-stealing.



Rust for Data Analysis II

Key crates (libraries):

- polars: fast **DataFrame** library backed by **Arrow**.
- arrow2: **Apache Arrow** in-memory columnar format.
- ndarray: N-dimensional arrays similar to NumPy.
- rayon: data-parallel iterators with work-stealing.



Rust for Data Analysis II

Key crates (libraries):

- polars: fast **DataFrame** library backed by **Arrow**.
- arrow2: **Apache Arrow** in-memory columnar format.
- ndarray: **N-dimensional arrays** similar to **NumPy**.
- rayon: **data-parallel** iterators with work-stealing.



Rust for Data Analysis II

Key crates (libraries):

- polars: fast **DataFrame** library backed by **Arrow**.
- arrow2: **Apache Arrow** in-memory columnar format.
- ndarray: **N-dimensional arrays** similar to **NumPy**.
- rayon: **data-parallel** iterators with work-stealing.



Missing Values

Missing Data

Missing values occur when **no data** is recorded for a variable in an observation. They can **bias results**, reduce **statistical power**, and cause **errors** in models if not handled properly.

- **MCAR** — Missing Completely at Random: **absence** is **independent** of any variable; **safest** to impute or drop.
- **MAR** — Missing at Random: **missingness** depends on **other observed** variables (e.g., age missing more often for elderly).
- **MNAR** — Missing Not at Random: **absence** depends on the **unobserved value** itself; **hardest** to correct.
- **Handling strategies:**
 - **Deletion:** remove rows/columns (listwise or pairwise).
 - **Mean / Median imputation:** replace with central tendency.
 - **KNN imputation:** borrow values from **similar observations**.
 - **Model-based:** predict missing values with regression or **random forest**.



Missing Values

Missing Data

Missing values occur when **no data** is recorded for a variable in an observation. They can **bias results**, reduce **statistical power**, and cause **errors** in models if not handled properly.

- **MCAR** — Missing Completely at Random: **absence** is **independent** of any variable; **safest** to impute or drop.
- **MAR** — Missing at Random: **missingness** depends on **other observed** variables (e.g., age missing more often for elderly).
- **MNAR** — Missing Not at Random: **absence** depends on the **unobserved value** itself; **hardest** to correct.
- **Handling strategies:**
 - Deletion: remove rows/columns (listwise or pairwise).
 - Mean / Median imputation: replace with central tendency.
 - KNN imputation: borrow values from **similar observations**.
 - Model-based: predict missing values with regression or **random forest**.



Missing Values

Missing Data

Missing values occur when **no data** is recorded for a variable in an observation. They can **bias results**, reduce **statistical power**, and cause **errors** in models if not handled properly.

- **MCAR** — Missing Completely at Random: **absence** is **independent** of any variable; **safest** to impute or drop.
- **MAR** — Missing at Random: **missingness** depends on **other observed** variables (e.g., age missing more often for elderly).
- **MNAR** — Missing Not at Random: **absence** depends on the **unobserved value** itself; **hardest** to correct.
- **Handling strategies:**
 - **Deletion:** remove rows/columns (listwise or pairwise).
 - **Mean / Median imputation:** replace with central tendency.
 - **KNN imputation:** borrow values from **similar observations**.
 - **Model-based:** predict missing values with regression or **random forest**.



Missing Values

Missing Data

Missing values occur when **no data** is recorded for a variable in an observation. They can **bias results**, reduce **statistical power**, and cause **errors** in models if not handled properly.

- **MCAR** — Missing Completely at Random: **absence** is **independent** of any variable; **safest** to impute or drop.
- **MAR** — Missing at Random: **missingness** depends on **other observed** variables (e.g., age missing more often for elderly).
- **MNAR** — Missing Not at Random: **absence** depends on the **unobserved value** itself; **hardest** to correct.
- **Handling strategies:**
 - **Deletion:** remove rows/columns (listwise or pairwise).
 - **Mean / Median imputation:** replace with central tendency.
 - **KNN imputation:** borrow values from **similar observations**.
 - **Model-based:** predict missing values with regression or **random forest**.



Missing Values

Missing Data

Missing values occur when **no data** is recorded for a variable in an observation. They can **bias results**, reduce **statistical power**, and cause **errors** in models if not handled properly.

- **MCAR** — Missing Completely at Random: **absence** is **independent** of any variable; **safest** to impute or drop.
- **MAR** — Missing at Random: **missingness** depends on **other observed** variables (e.g., age missing more often for elderly).
- **MNAR** — Missing Not at Random: **absence** depends on the **unobserved value** itself; **hardest** to correct.
- **Handling strategies:**
 - **Deletion**: remove rows/columns (listwise or pairwise).
 - **Mean / Median imputation**: replace with central tendency.
 - **KNN imputation**: borrow values from **similar observations**.
 - **Model-based**: **predict** missing values with regression or **random forest**.



Outlier Detection: IQR and Z-Score

IQR Method

$$IQR = Q_3 - Q_1$$

Bounds:

$$L = Q_1 - 1.5 \cdot IQR$$

$$U = Q_3 + 1.5 \cdot IQR$$

Observations outside $[L, U]$ are **outliers**.

Robust: does not assume **normality**.

Z-Score Method

$$z_i = \frac{x_i - \mu}{\sigma}$$

Values with $|z_i| > 3$ exceed **3 standard deviations** and are flagged as **outliers**.

Assumes: data is **approximately normal**.

When to use which?

Use **IQR** for **skewed** or **non-normal** data. Use **Z-score** when data is **approximately normal** and the **mean** and **standard deviation** are reliable.

Outlier Detection: IQR and Z-Score

IQR Method

$$IQR = Q_3 - Q_1$$

Bounds:

$$L = Q_1 - 1.5 \cdot IQR$$

$$U = Q_3 + 1.5 \cdot IQR$$

Observations outside $[L, U]$ are **outliers**.

Robust: does not assume **normality**.

Z-Score Method

$$z_i = \frac{x_i - \mu}{\sigma}$$

Values with $|z_i| > 3$ exceed **3 standard deviations** and are flagged as **outliers**.

Assumes: data is **approximately normal**.

When to use which?

Use **IQR** for **skewed** or **non-normal** data. Use **Z-score** when data is **approximately normal** and the **mean** and **standard deviation** are reliable.

Outlier Detection: IQR and Z-Score

IQR Method

$$IQR = Q_3 - Q_1$$

Bounds:

$$L = Q_1 - 1.5 \cdot IQR$$

$$U = Q_3 + 1.5 \cdot IQR$$

Observations outside $[L, U]$ are **outliers**.

Robust: does not assume **normality**.

Z-Score Method

$$z_i = \frac{x_i - \mu}{\sigma}$$

Values with $|z_i| > 3$ exceed **3 standard deviations** and are flagged as **outliers**.

Assumes: data is **approximately normal**.

When to use which?

Use **IQR** for **skewed** or **non-normal** data. Use **Z-score** when data is **approximately normal** and the **mean and standard deviation** are reliable.

Data Normalization

Feature Scaling

Normalization transforms features to a **common scale**, preventing **magnitude-dominated** behavior in algorithms sensitive to feature scale such as *KNN*, *SVM*, and *PCA*.

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Output range: $[0, 1]$.

Sensitive to outliers.

Z-Score Standardization

$$x' = \frac{x - \mu}{\sigma}$$

Output: $\mu' = 0$, $\sigma' = 1$.

Robust for **normal** distributions.

- **Robust Scaling**: uses **median** and **IQR** instead of mean/std; more resilient to **outliers**.

• **Log Transform**: $x' = \log(x + 1)$; reduces **variance** for **highly skewed**, **positive-valued** features.



Data Normalization

Feature Scaling

Normalization transforms features to a **common scale**, preventing **magnitude-dominated** behavior in algorithms sensitive to feature scale such as *KNN*, *SVM*, and *PCA*.

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Output range: $[0, 1]$.

Sensitive to outliers.

Z-Score Standardization

$$x' = \frac{x - \mu}{\sigma}$$

Output: $\mu' = 0$, $\sigma' = 1$.

Robust for **normal** distributions.

- **Robust Scaling:** uses **median** and **IQR** instead of mean/std; more resilient to **outliers**.
- **Log Transform:** $x' = \log(x + 1)$; reduces **right-skewness** for **heavy-tailed**, positive-valued features.



Data Normalization

Feature Scaling

Normalization transforms features to a **common scale**, preventing **magnitude-dominated** behavior in algorithms sensitive to feature scale such as *KNN*, *SVM*, and *PCA*.

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Output range: $[0, 1]$.

Sensitive to outliers.

Z-Score Standardization

$$x' = \frac{x - \mu}{\sigma}$$

Output: $\mu' = 0$, $\sigma' = 1$.

Robust for **normal** distributions.

- **Robust Scaling:** uses **median** and **IQR** instead of mean/std; more resilient to **outliers**.
- **Log Transform:** $x' = \log(x + 1)$; reduces **right-skewness** for **heavy-tailed**, positive-valued features.



Data Normalization

Feature Scaling

Normalization transforms features to a **common scale**, preventing **magnitude-dominated** behavior in algorithms sensitive to feature scale such as *KNN*, *SVM*, and *PCA*.

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Output range: $[0, 1]$.

Sensitive to outliers.

Z-Score Standardization

$$x' = \frac{x - \mu}{\sigma}$$

Output: $\mu' = 0$, $\sigma' = 1$.

Robust for **normal** distributions.

- **Robust Scaling:** uses **median** and **IQR** instead of mean/std; more resilient to **outliers**.
- **Log Transform:** $x' = \log(x + 1)$; reduces **right-skewness** for **heavy-tailed**, positive-valued features.



Pearson Correlation Coefficient I

Pearson's r

Pearson's r measures the **linear** association between two **continuous variables** X and Y , ranging from -1 to $+1$:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

- $r = +1$: perfect **positive linear** relationship.
- $r = -1$: perfect **negative linear** relationship.
- $r = 0$: no linear association (nonlinear dependence may still exist).



Pearson Correlation Coefficient I

Pearson's r

Pearson's r measures the **linear** association between two **continuous variables** X and Y , ranging from -1 to $+1$:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

- $r = +1$: perfect **positive linear** relationship.
- $r = -1$: perfect **negative linear** relationship.
- $r = 0$: **no linear** association (nonlinear dependence may still exist)



Pearson Correlation Coefficient I

Pearson's r

Pearson's r measures the **linear** association between two **continuous variables** X and Y , ranging from -1 to $+1$:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

- $r = +1$: perfect **positive linear** relationship.
- $r = -1$: perfect **negative linear** relationship.
- $r = 0$: **no linear** association (nonlinear dependence may still exist).



Pearson Correlation Coefficient II

- **Significance test:** $t = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}}$ follows a ***t*-distribution** with $n - 2$ degrees of freedom.
- **Assumptions:** linearity, bivariate normality, no outliers, and homoscedasticity.



Pearson Correlation Coefficient II

- **Significance test:** $t = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}}$ follows a *t-distribution* with $n - 2$ degrees of freedom.
- **Assumptions:** linearity, bivariate normality, no outliers, and homoscedasticity.



Polars: Modern DataFrame Library I

Polars

Polars is a **blazingly fast** DataFrame library written in **Rust** with a Python API, built on **Apache Arrow** for columnar **in-memory layout** and supporting **lazy** and **parallel** execution out of the box.

Advantages over pandas:

- **Lazy evaluation**: builds a **query plan** before execution, enabling automatic optimization.
- **Multithreaded**: uses all CPU cores by default with no extra configuration.
- **No index**: simpler mental model, fewer surprises.
- **SQL-like**: **composable**, **readable**, **SQL-like**, **transforms with predicate-pushdown support**.



Polars: Modern DataFrame Library I

Polars

Polars is a **blazingly fast** DataFrame library written in **Rust** with a Python API, built on **Apache Arrow** for columnar **in-memory layout** and supporting **lazy** and **parallel** execution out of the box.

Advantages over pandas:

- **Lazy evaluation**: builds a **query plan** before execution, enabling **automatic optimization**.
- **Multithreaded**: uses **all CPU cores** by default with no extra configuration.
- **No index**: simpler **mental model**, fewer surprises.
- **Expression API**: composable, readable, **chainable** transforms with predicate-pushdown support.



Polars: Modern DataFrame Library I

Polars

Polars is a **blazingly fast** DataFrame library written in **Rust** with a Python API, built on **Apache Arrow** for columnar **in-memory layout** and supporting **lazy** and **parallel** execution out of the box.

Advantages over pandas:

- **Lazy evaluation**: builds a **query plan** before execution, enabling **automatic optimization**.
- **Multithreaded**: uses **all CPU cores** by default with no extra configuration.
- **No index**: simpler **mental model**, fewer surprises.
- **Expression API**: composable, readable, **chainable** transforms with predicate-pushdown support.



Polars: Modern DataFrame Library I

Polars

Polars is a **blazingly fast** DataFrame library written in **Rust** with a Python API, built on **Apache Arrow** for columnar **in-memory layout** and supporting **lazy** and **parallel** execution out of the box.

Advantages over pandas:

- **Lazy evaluation**: builds a **query plan** before execution, enabling **automatic optimization**.
- **Multithreaded**: uses **all CPU cores** by default with no extra configuration.
- **No index**: simpler **mental model**, fewer surprises.
- **Expression API**: composable, readable, **chainable** transforms with predicate-pushdown support.



Polars: Modern DataFrame Library II

Key operations:

- `pl.scan_parquet()`: **lazy** Parquet scan; reads only required **columns and rows**.
- `.filter()` / `.select()`: efficient **predicate pushdown** to the storage layer.
- `.group_by().agg()`: fast **group aggregation** exploiting **SIMD** instructions.
- `.join()`: **hash join** and **sort-merge join** in parallel.



Polars: Modern DataFrame Library II

Key operations:

- `pl.scan_parquet()`: **lazy** Parquet scan; reads only required **columns and rows**.
- `.filter()` / `.select()`: efficient **predicate pushdown** to the storage layer.
- `.group_by().agg()`: fast **group aggregation** exploiting **SIMD** instructions.
- `.join()`: **hash join** and **sort-merge join** in parallel.



Polars: Modern DataFrame Library II

Key operations:

- `pl.scan_parquet()`: **lazy** Parquet scan; reads only required **columns and rows**.
- `.filter()` / `.select()`: efficient **predicate pushdown** to the storage layer.
- `.group_by().agg()`: fast **group aggregation** exploiting **SIMD** instructions.
- `.join()`: **hash join** and **sort-merge join** in parallel.



Polars: Modern DataFrame Library II

Key operations:

- `pl.scan_parquet()`: **lazy** Parquet scan; reads only required **columns and rows**.
- `.filter()` / `.select()`: efficient **predicate pushdown** to the storage layer.
- `.group_by().agg()`: fast **group aggregation** exploiting **SIMD** instructions.
- `.join()`: **hash join** and **sort-merge join** in parallel.



Outline

- 1 Tools and Process
- 2 Descriptive Analytics
- 3 Data Analysis



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/data-analysis-programming>

